Source: Patrick Hendry on Unsplash

OTEL YOU IT'S NOT JUST FOR BACKEND!

CARLY RICHMOND

ABOUT ME

- Developer Advocate & Manager @ elastic
- Frontend Engineer, Speaker & Blogger

SCAN ME

elastic

carlyrichmond.bsky.social

Source: Mark Stuckey on Unsplash

FIRST PORT OF CALL

Source: Dominik Lückmann on Unsplash


BACK OF THE QUEUE
Source: Rach Teo on Unsplash
elastic
carlyrichmond.bsky.social

Source: Meta Bakrate on Unsplash

REAL USER MONITORING

STILL ON THE RUM?

JavaScript | OpenTelemetry

https://opentelemetry.io/docs/languages/js/

New Chrome available

OpenTelemetry

Docs | Ecosystem | Status | Community | Blog | English | Search this site...

Docs / Language APIs & SDKs / JavaScript

JavaScript

A language-specific implementation of OpenTelemetry in JavaScript (for Node.js & the browser).

This is the OpenTelemetry JavaScript documentation. OpenTelemetry is an observability framework – an API, SDK, and tools that are designed to aid in the generation and collection of application telemetry data such as metrics, logs, and traces. This documentation is designed to help you understand how to get started using OpenTelemetry JavaScript.

Status and Releases

The current status of the major functional components for OpenTelemetry JavaScript is as follows:

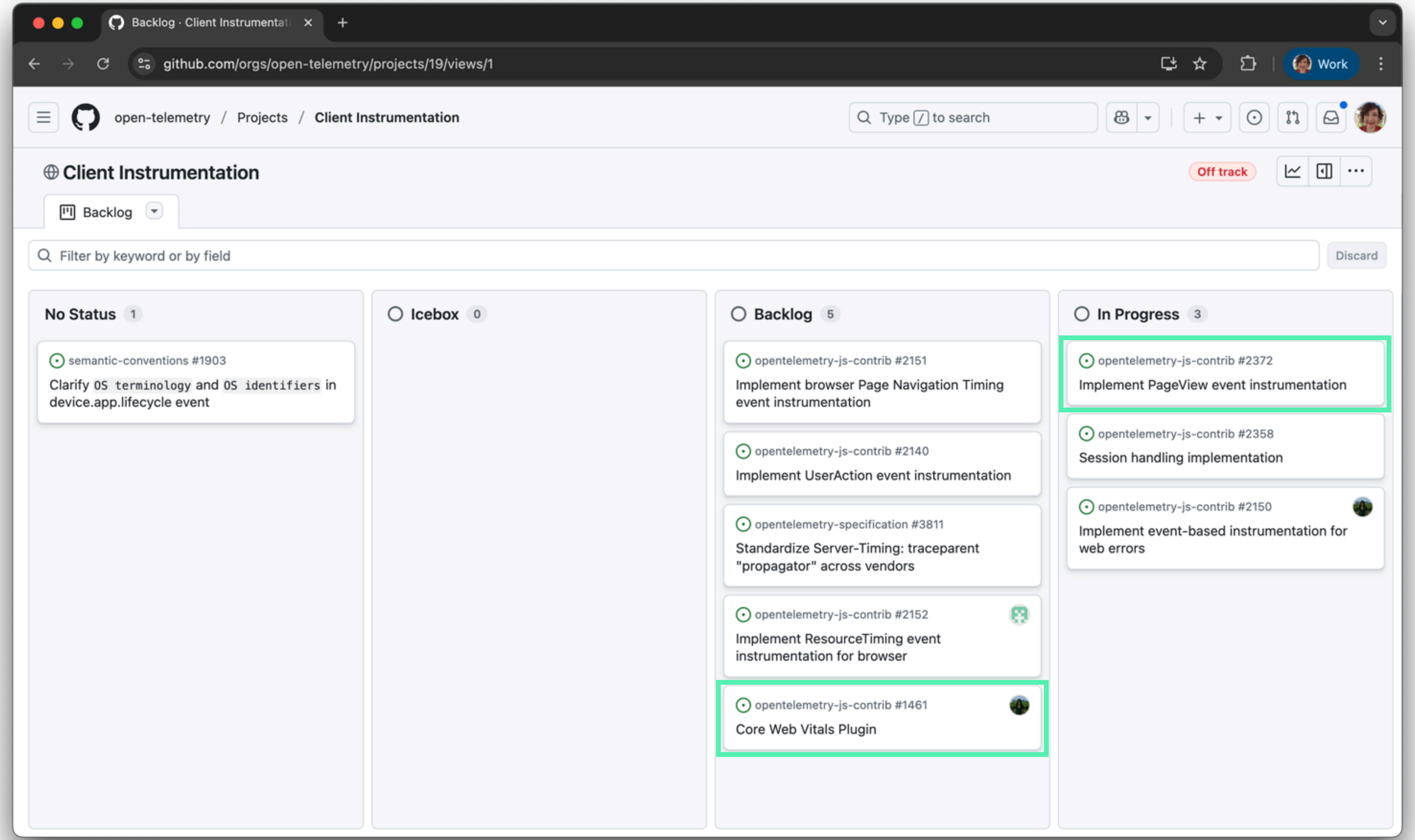
Traces	Metrics	Logs
Stable	Stable	Development

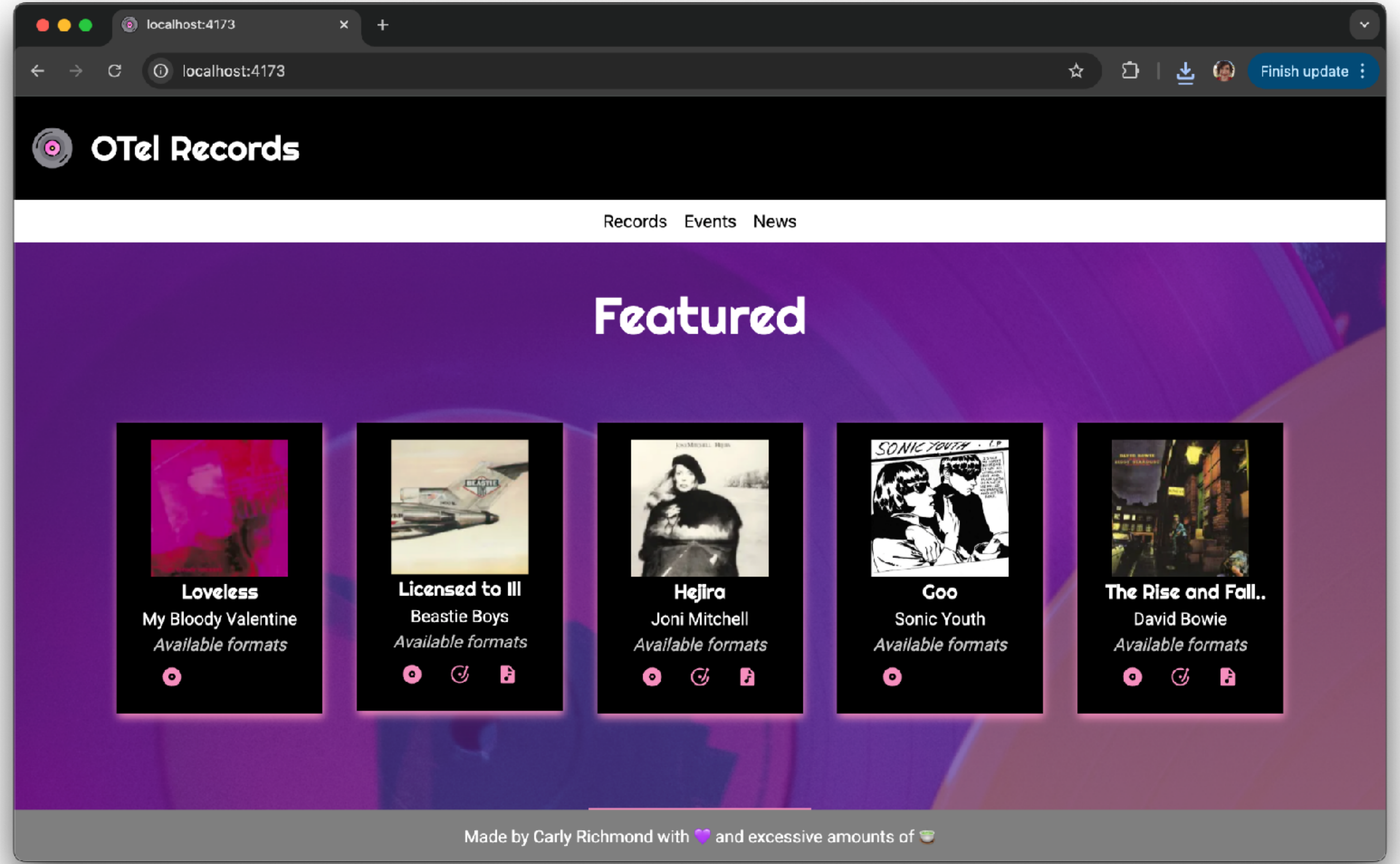
For releases, including the latest release, see Releases.

Warning

Client instrumentation for the browser is **experimental** and mostly **unspecified**. If you are interested in helping out, get in touch with the Client Instrumentation SIG.

View page source
Edit this page
Create documentation issue


Backlog · Client Instrumentati
github.com/orgs/open-telemetry/projects/19/views/1
Work
open-telemetry / Projects / Client Instrumentation
Type / to search
Client Instrumentation
Off track
Backlog
Filter by keyword or by field
Discard
No Status 1
semantic-conventions #1903
Clarify OS terminology and OS identifiers in device.app.lifecycle event
Icebox 0
Backlog 5
opentelemetry-js-contrib #2151
Implement browser Page Navigation Timing event instrumentation
opentelemetry-js-contrib #2140
Implement UserAction event instrumentation
opentelemetry-specification #3811
Standardize Server-Timing: traceparent "propagator" across vendors
opentelemetry-js-contrib #2152
Implement ResourceTiming event instrumentation for browser
opentelemetry-js-contrib #1461
Core Web Vitals Plugin
In Progress 3
opentelemetry-js-contrib #2372
Implement PageView event instrumentation
opentelemetry-js-contrib #2358
Session handling implementation
opentelemetry-js-contrib #2150
Implement event-based instrumentation for web errors

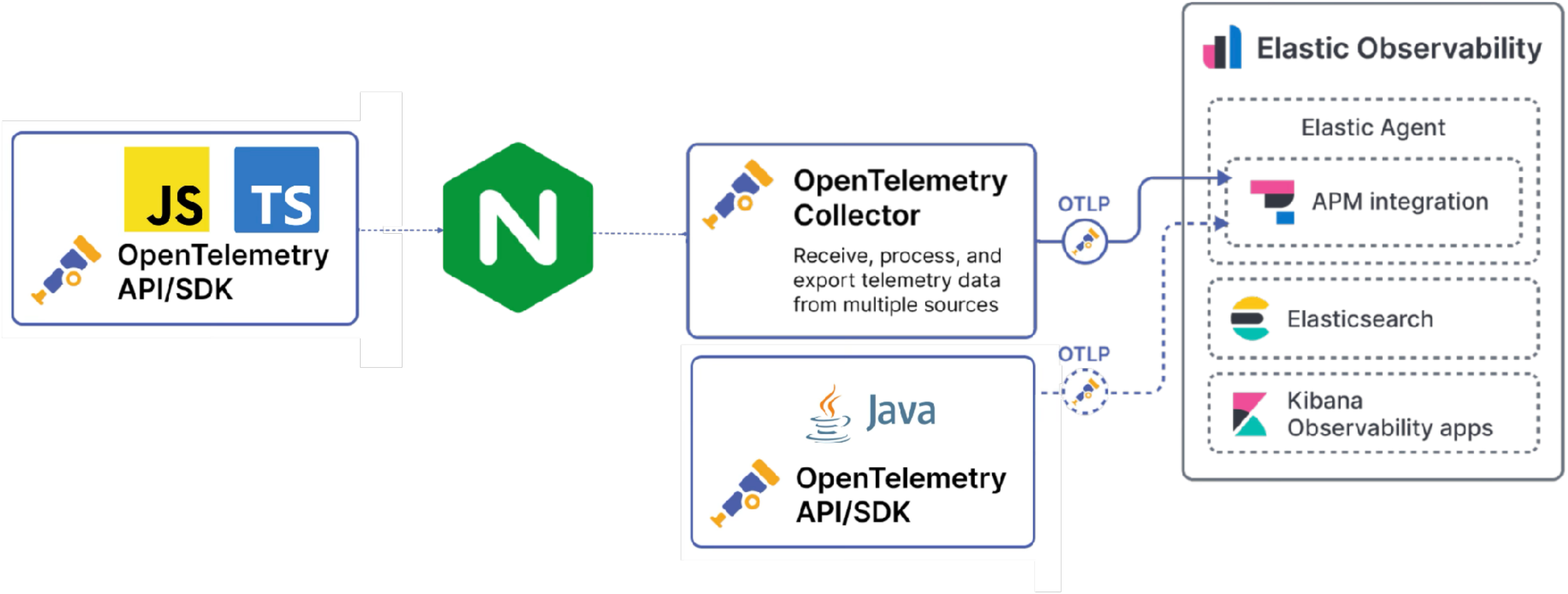


SCAN ME

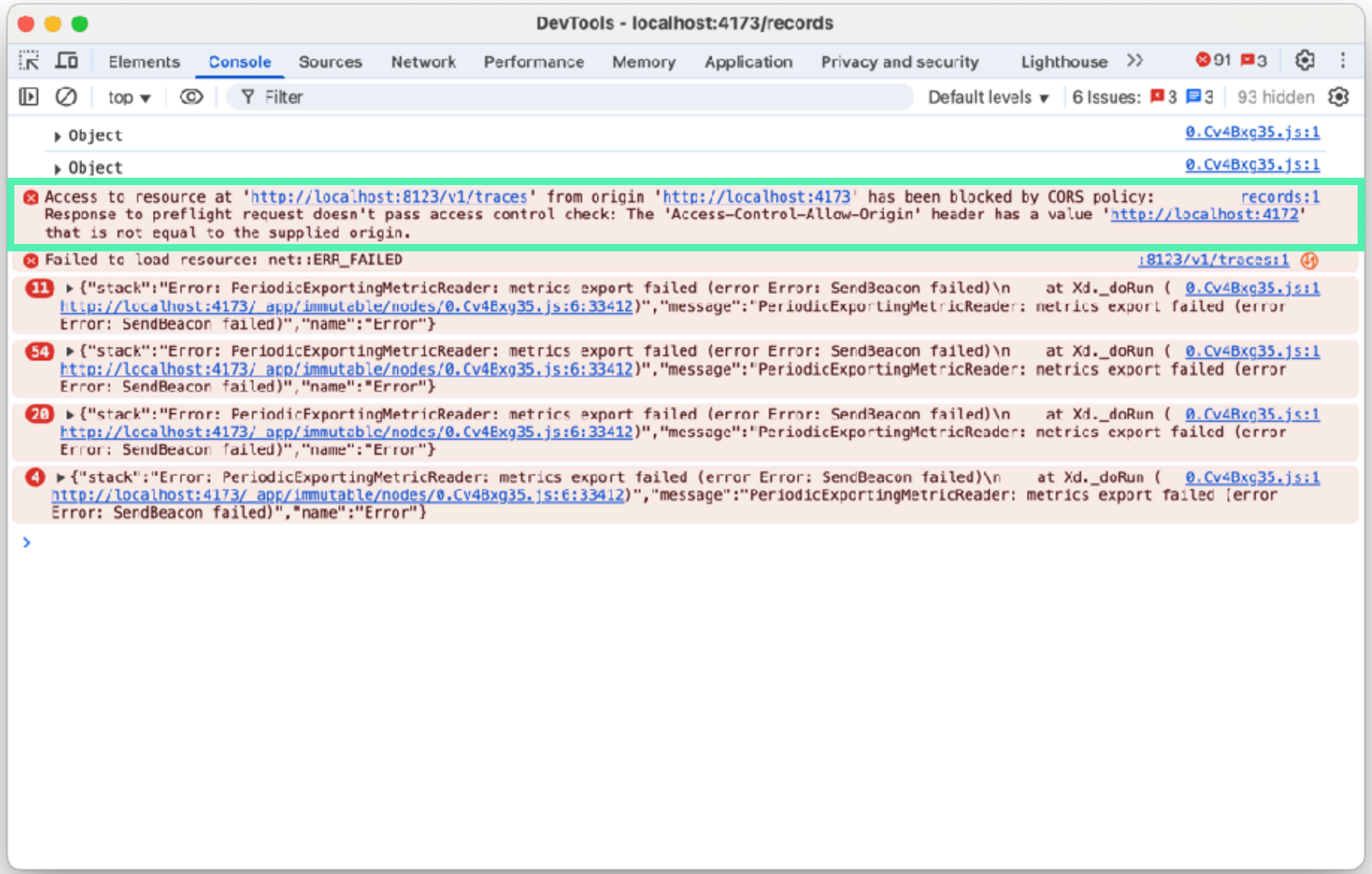
elastic

carlyrichmond.bsky.social

OTEL WEB ARCHITECTURE



CUE CORS HELL!



NGINX PROXY

nginx.conf

```
events {}

http {

  server {

    listen 8123;

    location /v1/traces {
      proxy_pass http://host.docker.internal:4318;
      # Apply CORS headers to ALL responses, including POST
      add_header 'Access-Control-Allow-Origin' 'http://localhost:4173' always;
      add_header 'Access-Control-Allow-Methods' 'POST, OPTIONS' always;
      add_header 'Access-Control-Allow-Headers' 'Content-Type' always;
      add_header 'Access-Control-Allow-Credentials' 'true' always;

      # Preflight requests receive a 204 No Content response
      if ($request_method = OPTIONS) {
        return 204;
      }
    }
  }
}
```

frontend.tracer.ts

```typescript
/* OpenTelemetry Frontend packages */

// See https://github.com/carlyrichmond/otel-record-store for import list

// Define resource metadata for the service, used by exporters (Elastic requires service.name)
const SERVICE_NAME = 'records-ui-web';

const detectedResources = detectResources({ detectors: [browserDetector] });
let resource = resourceFromAttributes({
    [ATTR_SERVICE_NAME]: SERVICE_NAME,
    'service.version': 1,
    'deployment.environment': 'dev'
});
resource = resource.merge(detectedResources);

// Configure logging to send to the collector via nginx
const logExporter = new OTLPLogExporter({
    url: LOGS_URL // nginx proxy
});

const loggerProvider = new LoggerProvider({
    resource: resource,
    processors: [new BatchLogRecordProcessor(logExporter)]
});

logs.setGlobalLoggerProvider(loggerProvider);
```

Source: Ruben. on Unsplash

Logs

Metrics

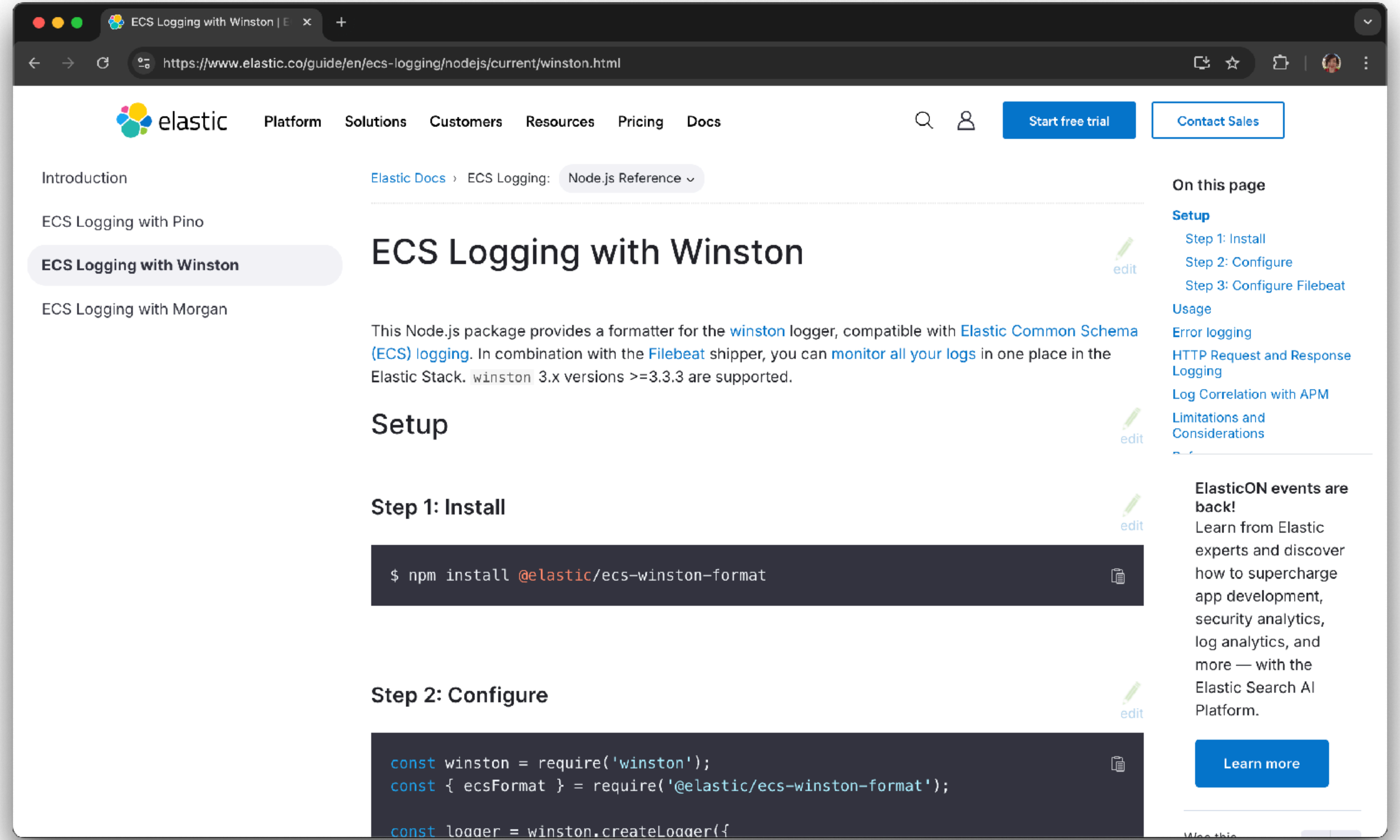
Traces

Source: Ruben. on Unsplash

Logs

Metrics

Traces



ECS Logging with Winston | E

https://www.elastic.co/guide/en/ecs-logging/nodejs/current/winston.html

elastic Platform Solutions Customers Resources Pricing Docs Start free trial Contact Sales

Introduction

ECS Logging with Pino

ECS Logging with Winston

ECS Logging with Morgan

Elastic Docs › ECS Logging: Node.js Reference

ECS Logging with Winston

edit

This Node.js package provides a formatter for the winston logger, compatible with Elastic Common Schema (ECS) logging. In combination with the Filebeat shipper, you can monitor all your logs in one place in the Elastic Stack. `winston` 3.x versions >=3.3.3 are supported.

Setup

edit

Step 1: Install

edit

```
$ npm install @elastic/ecs-winston-format
```

Step 2: Configure

edit

```
const winston = require('winston');
const { ecsFormat } = require('@elastic/ecs-winston-format');

const logger = winston.createLogger({
```

On this page

Setup

Step 1: Install

Step 2: Configure

Step 3: Configure Filebeat

Usage

Error logging

HTTP Request and Response Logging

Log Correlation with APM

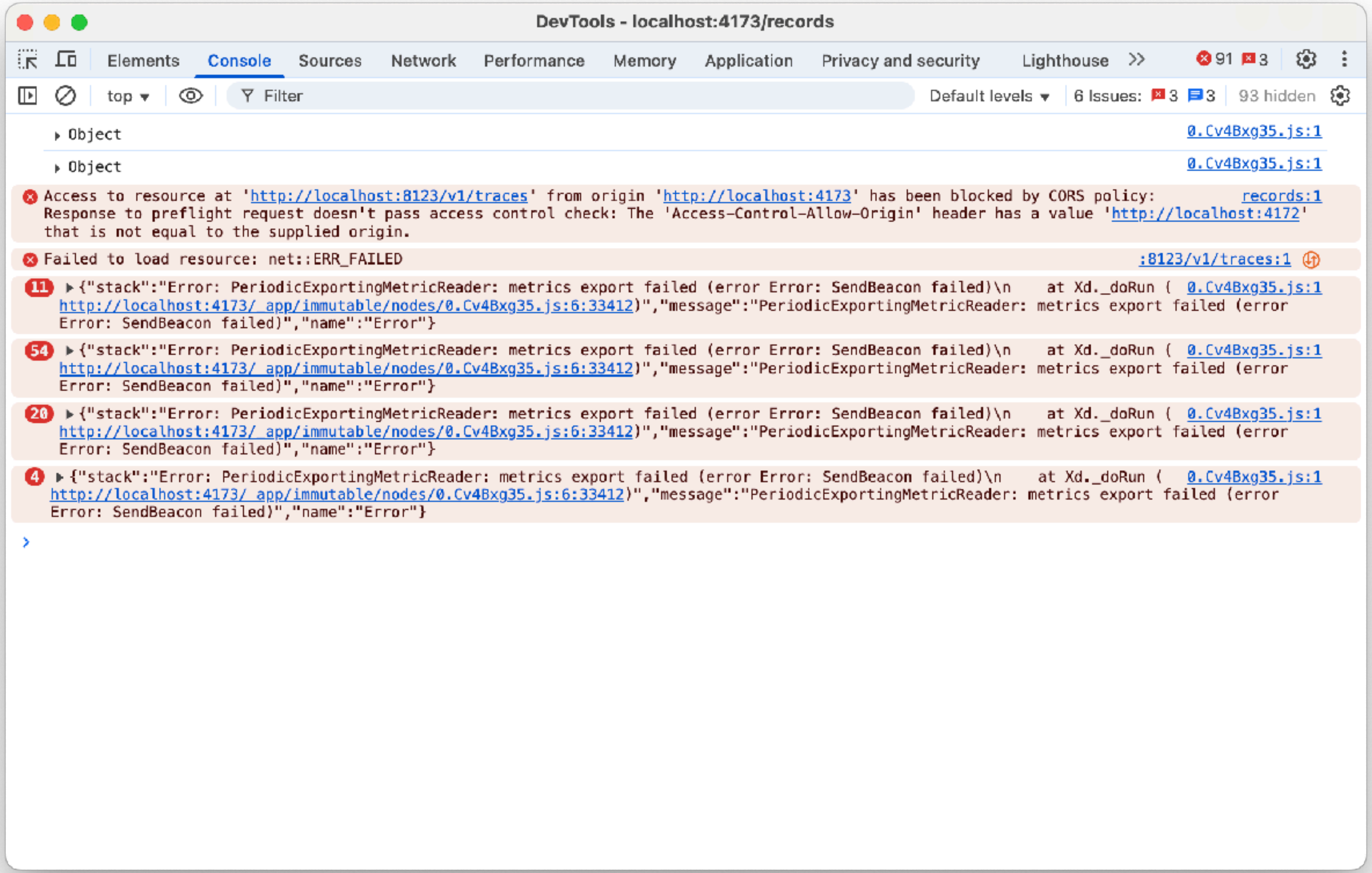
Limitations and Considerations

ElasticON events are back!

Learn from Elastic experts and discover how to supercharge app development, security analytics, log analytics, and more — with the Elastic Search AI Platform.

Learn more

DEVTOOLS CONSOLE ISN'T VISIBLE!



frontend.tracer.ts

```typescript
/* OpenTelemetry Frontend packages */

// See https://github.com/carlyrichmond/otel-record-store for import list

// Define resource metadata for the service, used by exporters (Elastic requires service.name)
const SERVICE_NAME = 'records-ui-web';

const detectedResources = detectResources({ detectors: [browserDetector] });
let resource = resourceFromAttributes({
    [ATTR_SERVICE_NAME]: SERVICE_NAME,
    'service.version': 1,
    'deployment.environment': 'dev'
});
resource = resource.merge(detectedResources);

// Configure logging to send to the collector via nginx
const logExporter = new OTLPLogExporter({
    url: LOGS_URL // nginx proxy
});

const loggerProvider = new LoggerProvider({
    resource: resource,
    processors: [new BatchLogRecordProcessor(logExporter)]
});

logs.setGlobalLoggerProvider(loggerProvider);

```

```
    'service.version': 1,
    'deployment.environment': 'dev'
});
resource = resource.merge(detectedResources);

// Configure logging to send to the collector via nginx
const logExporter = new OTLPLogExporter({
    url: LOGS_URL // nginx proxy
});

const loggerProvider = new LoggerProvider({
    resource: resource,
    processors: [new BatchLogRecordProcessor(logExporter)]
});

logs.setGlobalLoggerProvider(loggerProvider);

const logger = logs.getLogger('default', '1.0.0');
logger.emit({
    severityNumber: SeverityNumber.INFO,
    severityText: 'INFO',
    body: 'Logger initialized'
});

// Configure the OTLP exporter to talk to the collector via nginx
const exporter = new OTLPTraceExporter({
    url: TRACE_URL // nginx proxy
});

```

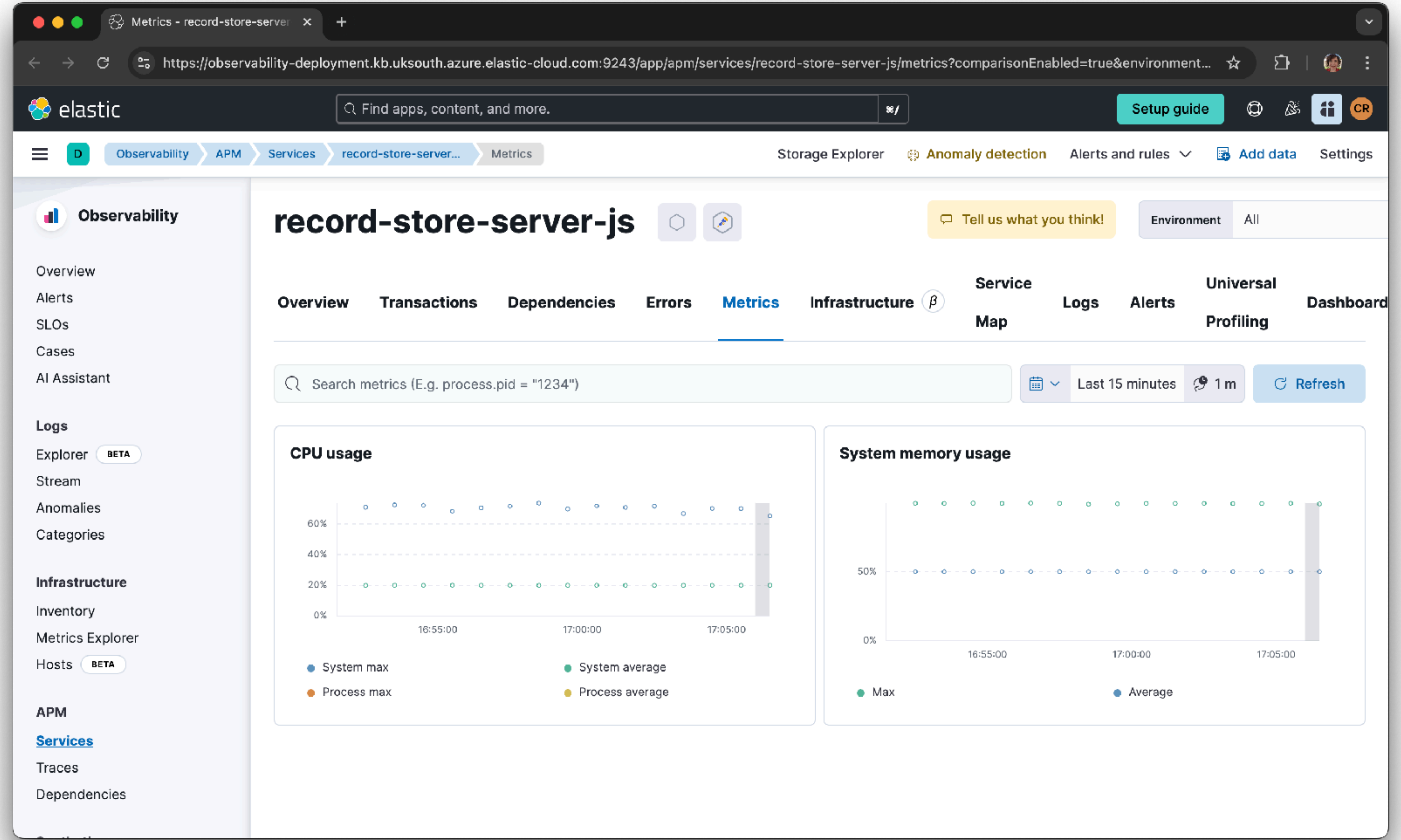

otel-record-store
Discover
Give feedback
AI Assistant
Try ES|QL
Data sets
Inspect
Alerts
Save
Data view
All logs
service.name : "records-ui-web"
Last 15 minutes
Refresh
Search field names
Auto interval
Breakdown by log.level
Selected fields 3
body.text
severity_text
severity_number
Popular fields 3
body.text
severity_number
severity_text
Available fields 133
@timestamp
agent.name
agent.version
attributes.error.exception.handled
attributes.error.grouping_key
attributes.error.grouping_name
attributes.error.id
attributes.event.name
attributes.exception.message
attributes.exception.type
attributes.exit_code
attributes.http.conn.done.addr
Add a field
INFO
(missing value)
17:17 17:18 17:19 17:20 17:21 17:22 17:23 17:24 17:25 17:26 17:27 17:28 17:29 17:30 17:31 17:32
September 5, 2025
Sep 5, 2025 @ 17:17:13.637 - Sep 5, 2025 @ 17:32:13.637 (interval: Auto - 30 seconds)
Documents (451)
Patterns
Field statistics
Columns 4
Sort fields 1
Actions
@timestamp
body.text
severity_text
severity_number
Sep 5, 2025 @ 17:32:03.305 Client Telemetry Initialised INFO 9
Sep 5, 2025 @ 17:32:03.296 Logger initialized INFO 9
Sep 5, 2025 @ 17:31:58.489 Client Telemetry Initialised INFO 9
Sep 5, 2025 @ 17:31:58.480 Logger initialized INFO 9
Sep 5, 2025 @ 17:31:57.736 Client Telemetry Initialised INFO 9
Sep 5, 2025 @ 17:31:57.727 Logger initialized INFO 9
Sep 5, 2025 @ 17:31:53.188 Client Telemetry Initialised INFO 9

Source: Ruben. on Unsplash

Logs

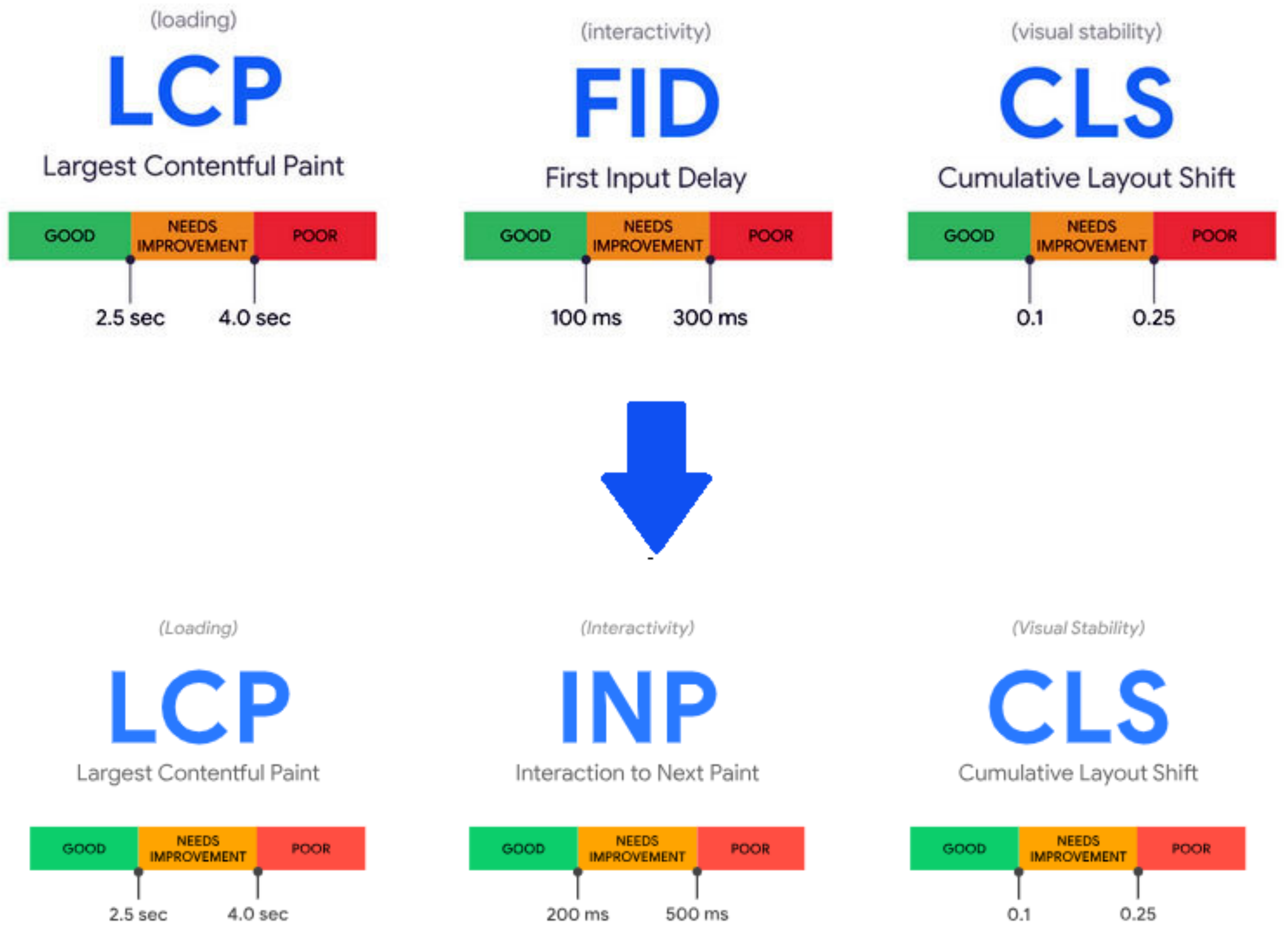
Metrics

Traces


Metrics - record-store-server
https://observability-deployment.kb.uksouth.azure.elastic-cloud.com:9243/app/apm/services/record-store-server-js/metrics?comparisonEnabled=true&environment...
elastic
Find apps, content, and more.
Setup guide
CR
D
Observability
APM
Services
record-store-server...
Metrics
Storage Explorer
Anomaly detection
Alerts and rules
Add data
Settings
Observability
Overview
Alerts
SLOs
Cases
AI Assistant
Logs
Explorer
BETA
Stream
Anomalies
Categories
Infrastructure
Inventory
Metrics Explorer
Hosts
BETA
APM
Services
Traces
Dependencies
record-store-server-js
Tell us what you think!
Environment
All
Overview
Transactions
Dependencies
Errors
Metrics
Infrastructure
Service Map
Logs
Alerts
Universal Profiling
Dashboard
Search metrics (E.g. process.pid = "1234")
Last 15 minutes
1 m
Refresh
CPU usage
60%
40%
20%
0%
16:55:00
17:00:00
17:05:00
System max
System average
Process max
Process average
System memory usage
50%
0%
16:55:00
17:00:00
17:05:00
Max
Average

https://web.dev/explore/learn-core-web-vitals

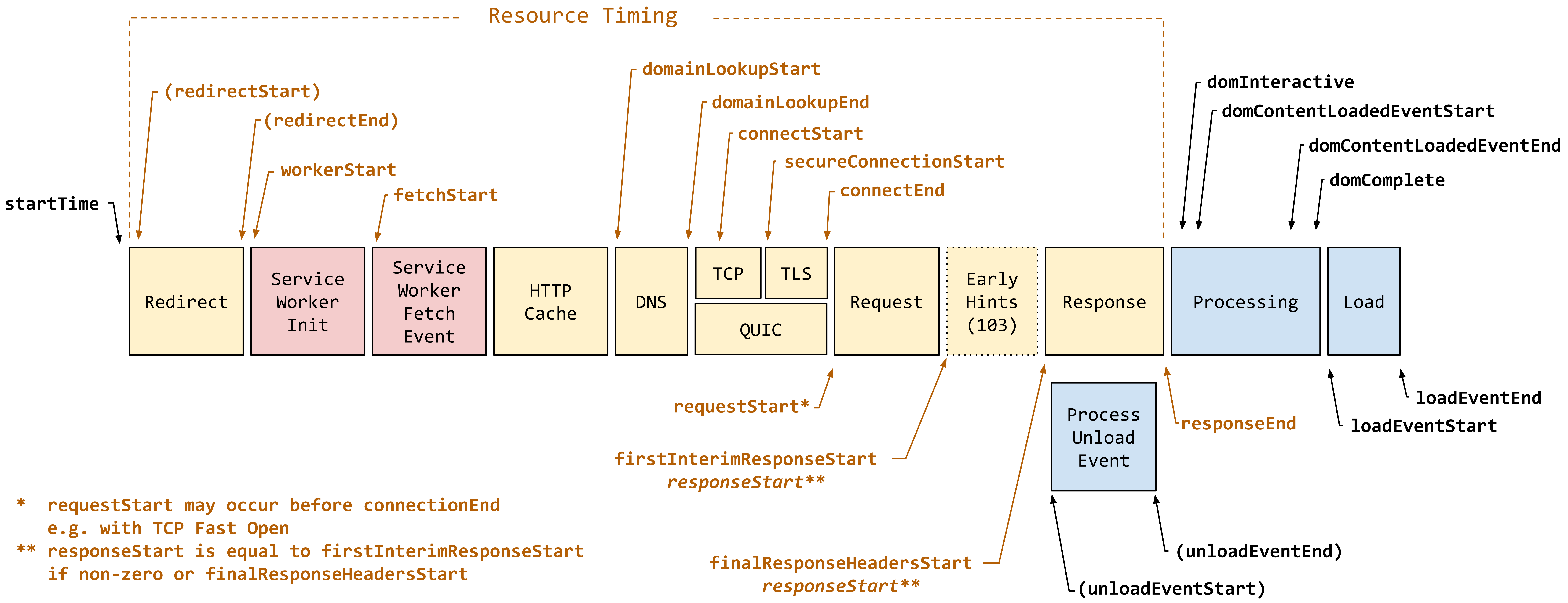
GOOGLE CORE WEB VITALS



https://web.dev/explore/learn-core-web-vitals

OTHER VITALS

PAGE LOAD

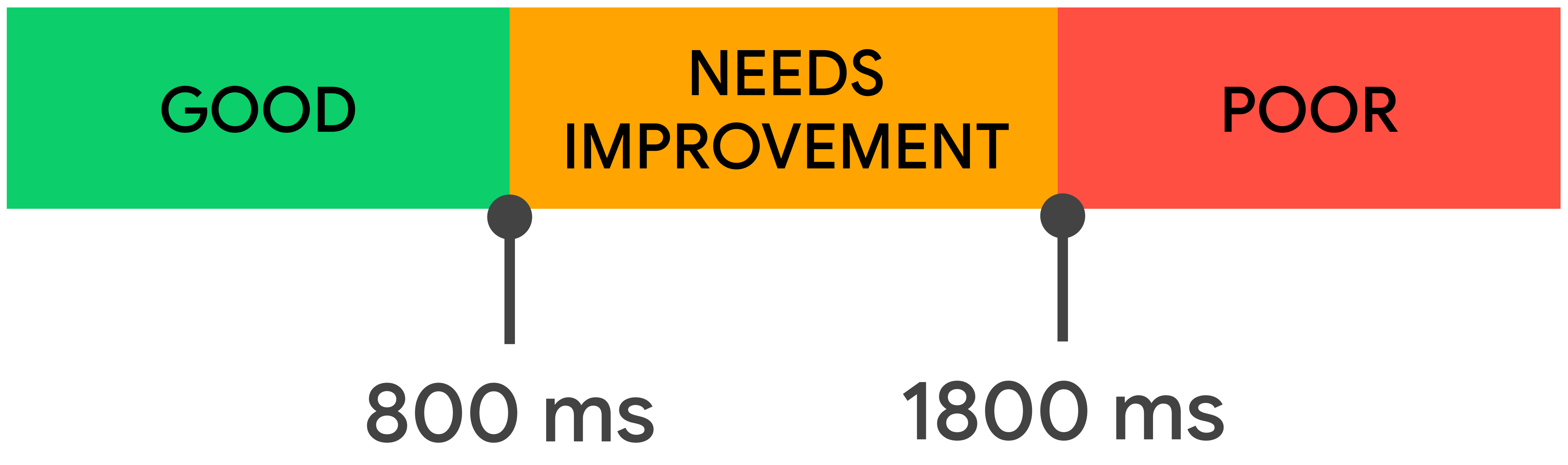


https://web.dev/articles/ttfb

TTFB

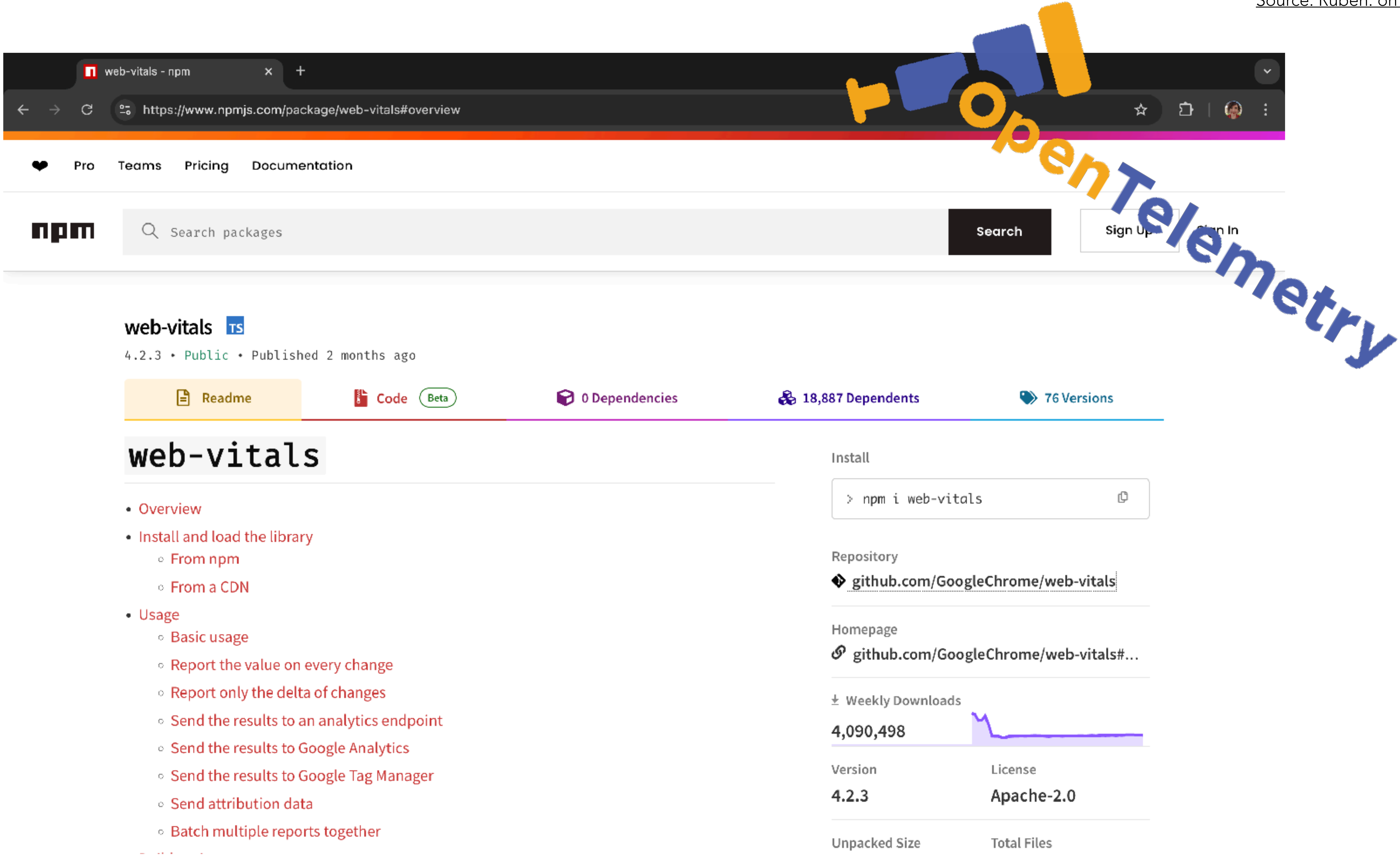
TIME TO FIRST BYTE

The time between the request for a resource being initiated and when the first byte of a response begins to arrive.



https://web.dev/articles/ttfb

Source: Ruben. on Unsplash



web-vitals.instrumentation.ts

```typescript
/* Web Vitals Frontend package*/

// See https://github.com/carlyrichmond/otel-record-store for import list

type CWVMetric = LCPMetric | CLSMetric | INPMetric | TTFBMetric | FCPMetric;

export class WebVitalsInstrumentation extends InstrumentationBase {

    private cwvMeter: Meter;

    /* Core Web Vitals Measures */
    private lcp: ObservableGauge;
    private cls: ObservableGauge;
    private inp: ObservableGauge;
    private ttfb: ObservableGauge;
    private fcp: ObservableGauge;

    constructor(config: InstrumentationConfig, resource: Resource) {
        super('WebVitalsInstrumentation', '1.0', config);

        const myServiceMeterProvider = this.generateMeterForResource(resource);
        metrics.setGlobalMeterProvider(myServiceMeterProvider);

        this.cwvMeter = metrics.getMeter('core-web-vitals', '1.0.0');

        // Initialising CWV metric instruments
        this.lcp = this.cwvMeter.createObservableGauge('lcp', { unit: 'ms', description: 'Largest Contentful Paint' });
        this.cls = this.cwvMeter.createObservableGauge('cls', { description: 'Cumulative Layout Shift' });
        this.inp = this.cwvMeter.createObservableGauge('inp', { unit: 'ms', description: 'Interaction to Next Paint' });
        this.ttfb = this.cwvMeter.createObservableGauge('ttfb', { unit: 'ms', description: 'Time to First Byte' });
```

```
    constructor(config: InstrumentationConfig, resource: Resource) {
        super('WebVitalsInstrumentation', '1.0', config);

        const myServiceMeterProvider = this.generateMeterForResource(resource);
        metrics.setGlobalMeterProvider(myServiceMeterProvider);

        this.cwvMeter = metrics.getMeter('core-web-vitals', '1.0.0');

        // Initialising CWV metric instruments
        this.lcp = this.cwvMeter.createObservableGauge('lcp', { unit: 'ms', description: 'Largest Contentful Paint' });
        this.cls = this.cwvMeter.createObservableGauge('cls', { description: 'Cumulative Layout Shift' });
        this.inp = this.cwvMeter.createObservableGauge('inp', { unit: 'ms', description: 'Interaction to Next Paint' });
        this.ttfb = this.cwvMeter.createObservableGauge('ttfb', { unit: 'ms', description: 'Time to First Byte' });
        this.fcp = this.cwvMeter.createObservableGauge('fcp', { unit: 'ms', description: 'First Contentful Paint' });
    }

    protected init(): InstrumentationModuleDefinition | InstrumentationModuleDefinition[] | void {}

    // Creating Meter Provider factory to send metrics via OTLP
    private generateMeterForResource(resource: Resource) {
        const metricReader = new PeriodicExportingMetricReader({
            exporter: new OTLPMetricExporter({
                url: METRICS_URL //nginx proxy
            }),
            // Default is 60000ms (60 seconds).
            // Set to 10 seconds for demo purposes only.
            exportIntervalMillis: 10000
        });

        return new MeterProvider({
            resource: resource,
            readers: [metricReader]
        });
```

```
    enable() {
        // Capture Largest Contentful Paint
        onLCP(
            (metric) => {
                this.lcp.addCallback((result) => {
                    this.sendMetric(metric, result);
                });
            },
            { reportAllChanges: true }
        );

        // Capture Cumulative Layout Shift
        onCLS(
            (metric) => {
                this.cls.addCallback((result) => {
                    this.sendMetric(metric, result);
                });
            },
            { reportAllChanges: true }
        );

        // Capture Interaction to Next Paint
        onINP(
            (metric) => {
                this.inp.addCallback((result) => {
                    this.sendMetric(metric, result);
                });
            },
            { reportAllChanges: true }
        );

        // Time to First Byte
        onTTFB(
```

```
        });
      },
      { reportAllChanges: true }
    );

    // First Contentful Paint
    onFCP(
      (metric) => {
        this.fcp.addCallback((result) => {
          this.sendMetric(metric, result);
        });
      },
      { reportAllChanges: true }
    );
  }

  private sendMetric(metric: CWVMetric, result: ObservableResult<Attributes>): void {
    const now = hrTime();

    const attributes = {
      startTime: now,
      'web_vital.name': metric.name,
      'web_vital.id': metric.id,
      'web_vital.navigationType': metric.navigationType,
      'web_vital.delta': metric.delta,
      'web_vital.value': metric.value,
      'web_vital.rating': metric.rating,
      // metric specific attributes
      'web_vital.entries': JSON.stringify(metric.entries)
    };

    result.observe(metric.value, attributes);
  }
}
```

OTel Core Web Vitals Summary

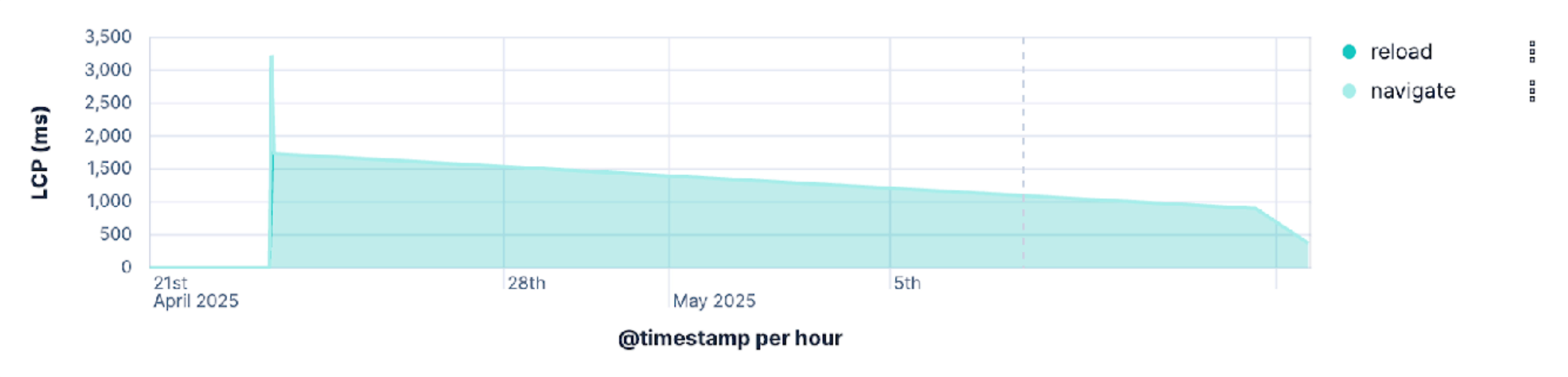
This dashboard shows the results of Core Web Vitals captured from the OTel record store application using the web-vitals JS library and OpenTelemetry JavaScript SDK.

Full instrumentation code available here.

Largest Contentful Paint (LCP)

good

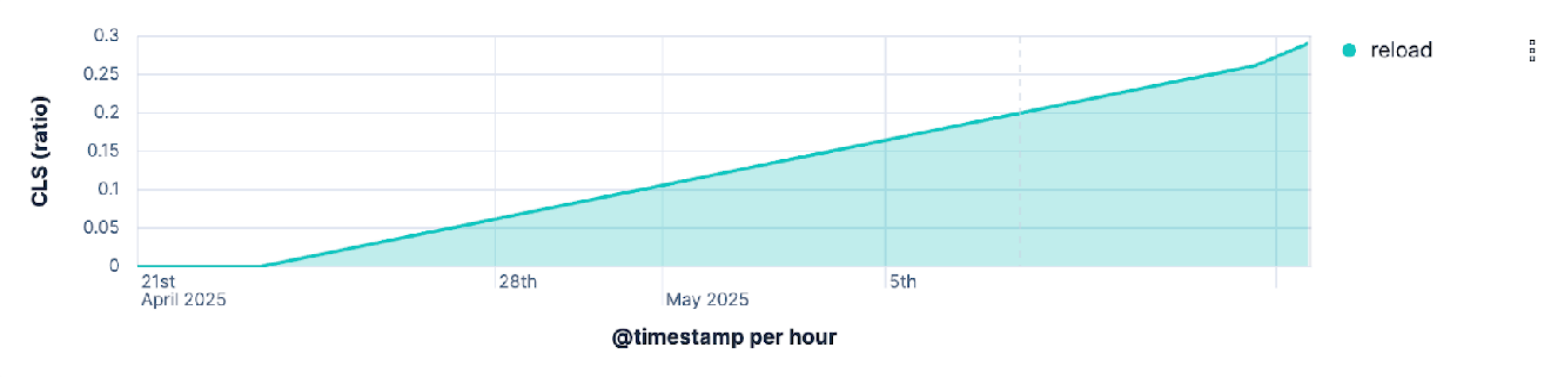
372



Cumulative Layout Shift (CLS)

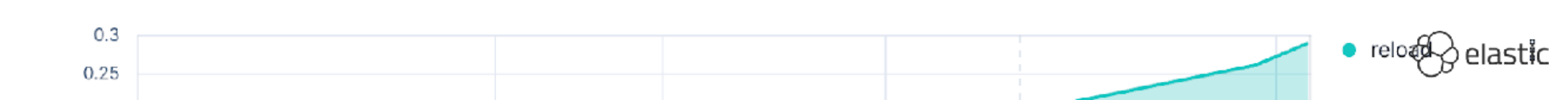
poor

0.291



Interaction to Next Paint (INP)

Exit full screen



elastic

Source: Ruben. on Unsplash


D
otel-record-store
Applications
...
records-ui-web
Transactions
HTTP GET
Give feedback
AI Assistant
CR
Give feedback
Alerts
Settings
Add data
Latency distribution
38 total transactions
Click and drag to select a range
10
1
Transactions
95p
Current sample
8 ms
10 ms
20 ms
30 ms
40 ms
50 ms
60 ms
80 ms
100 ms
200 ms
300 ms
400 ms
500 ms
600 ms
800 ms
1 s
2 s
3 s
Latency
All transactions
Trace sample
1 of 38
Investigate
View full trace
17 seconds ago | 39 ms (100% of trace)
Timeline
Metadata
Logs
Show critical path
Services
records-ui-web
record-store-server-java
0 ms
5.0 ms
10 ms
15 ms
20 ms
25 ms
30 ms
35 ms
39 ms
1
HTTP GET 39 ms
1
HTTP 2xx GET /records/featured 2.1 ms
GET featured 582 µs

frontend.tracer.ts

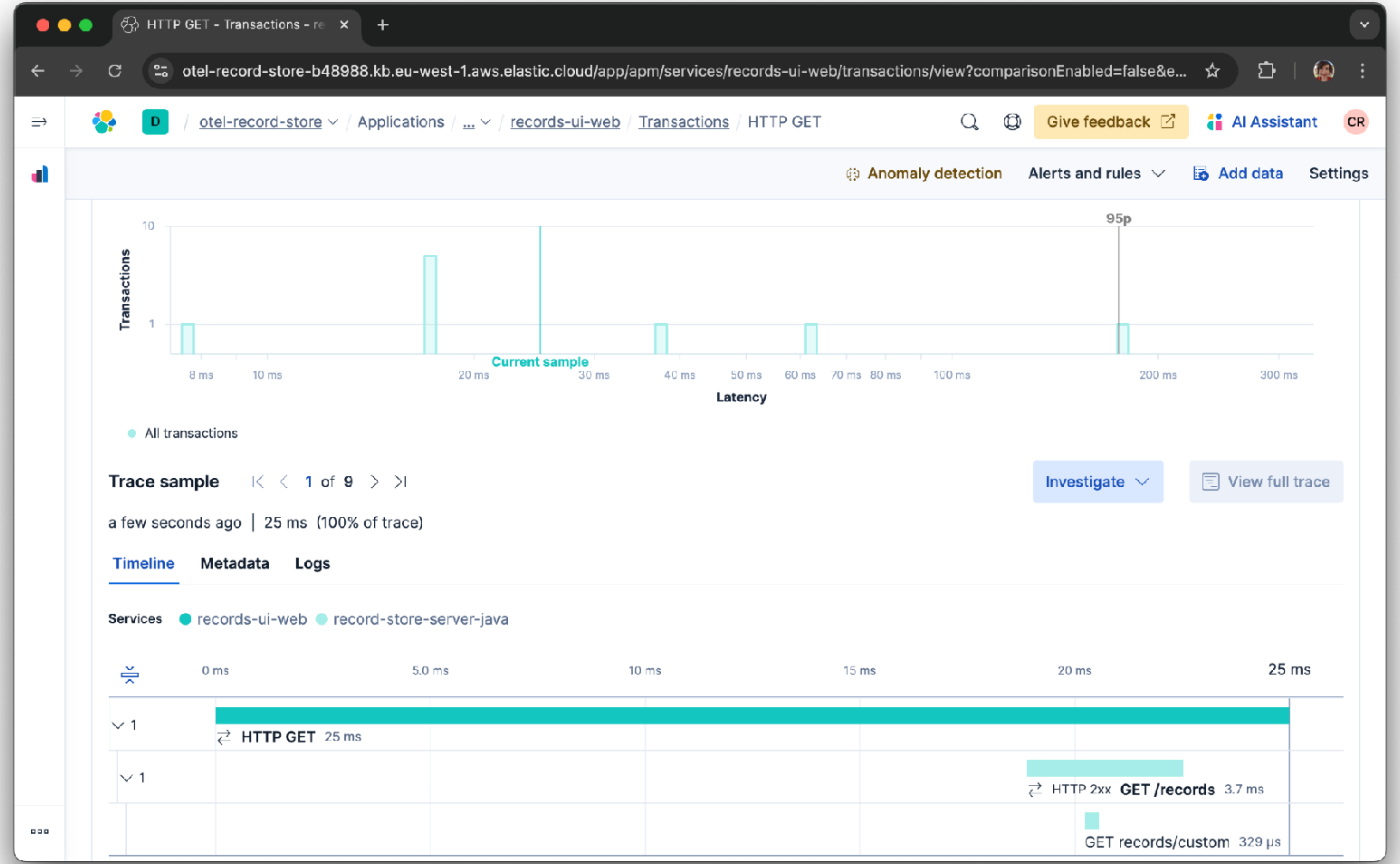
```typescript
/* OpenTelemetry Frontend packages */

// See https://github.com/carlyrichmond/otel-record-store for import list

// Define resource metadata for the service, used by exporters (Elastic requires service.name)
const SERVICE_NAME = 'records-ui-web';

const detectedResources = detectResources({ detectors: [browserDetector] });
let resource = resourceFromAttributes({
    [ATTR_SERVICE_NAME]: SERVICE_NAME,
    'service.version': 1,
    'deployment.environment': 'dev'
});
resource = resource.merge(detectedResources);

// Configure logging to send to the collector via nginx
const logExporter = new OTLPLogExporter({
    url: LOGS_URL // nginx proxy
});

const loggerProvider = new LoggerProvider({
    resource: resource,
    processors: [new BatchLogRecordProcessor(logExporter)]
});

logs.setGlobalLoggerProvider(loggerProvider);

```

```
const logger = logs.getLogger('default', '1.0.0');
logger.emit({
    severityNumber: SeverityNumber.INFO,
    severityText: 'INFO',
    body: 'Logger initialized'
});

// Configure the OTLP exporter to talk to the collector via nginx
const exporter = new OTLPTraceExporter({
    url: TRACE_URL // nginx proxy
});

// Instantiate the trace provider and inject the resource
const provider = new WebTracerProvider({
    resource: resource,
    spanProcessors: [
        // Send each completed span through the exporter
        new BatchSpanProcessor(exporter),
        new BatchSpanProcessor(new ConsoleSpanExporter())
    ]
});

// Register the provider with propagation and set up the async context manager for spans
provider.register({
    contextManager: new ZoneContextManager(),
    propagator: new CompositePropagator({
        propagators: [new W3CBaggagePropagator(), new W3CTraceContextPropagator()]
    })
});
```

```
// Instantiate the trace provider and inject the resource
const provider = new WebTracerProvider({
    resource: resource,
    spanProcessors: [
        // Send each completed span through the exporter
        new BatchSpanProcessor(exporter),
        new BatchSpanProcessor(new ConsoleSpanExporter())
    ]
});

// Register the provider with propagation and set up the async context manager for spans
provider.register({
    contextManager: new ZoneContextManager(),
    propagator: new CompositePropagator({
        propagators: [new W3CBaggagePropagator(), new W3CTraceContextPropagator()]
    })
});

export class ClientTelemetry {
    private static instance: ClientTelemetry;
    private initialized = false;

    private constructor() {}

    // Obtain singleton instance of provider
    public static getInstance(): ClientTelemetry {
        if (!ClientTelemetry.instance) {
            ClientTelemetry.instance = new ClientTelemetry();
            ClientTelemetry.instance.start();
```

WE ALL HAVE BAGGAGE!

Source: engin akyurt on Unsplash



elastic

carlyrichmond.bsky.social

frontend.tracer.ts

```typescript
/* OpenTelemetry Frontend packages */

// See https://github.com/carlyrichmond/otel-record-store for import list

// Define resource metadata for the service, used by exporters (Elastic requires service.name)
const SERVICE_NAME = 'records-ui-web';

const detectedResources = detectResources({ detectors: [browserDetector] });
let resource = resourceFromAttributes({
    [ATTR_SERVICE_NAME]: SERVICE_NAME,
    'service.version': 1,
    'deployment.environment': 'dev'
});
resource = resource.merge(detectedResources);

// Configure logging to send to the collector via nginx
const logExporter = new OTLPLogExporter({
    url: LOGS_URL // nginx proxy
});

const loggerProvider = new LoggerProvider({
    resource: resource,
    processors: [new BatchLogRecordProcessor(logExporter)]
});

logs.setGlobalLoggerProvider(loggerProvider);

```

```
    // Initializer
    public start() {
      if (!this.initialized) {
        // Enable automatic span generation for document load and user click interactions
        registerInstrumentations({
          instrumentations: [
            // Automatically tracks when the document loads
            new DocumentLoadInstrumentation({
              ignoreNetworkEvents: false,
              ignorePerformancePaintEvents: false
            }),
            getWebAutoInstrumentations({
              '@opentelemetry/instrumentation-fetch': {
                propagateTraceHeaderCorsUrls: /.*/,
                clearTimingResources: true
              },
              '@opentelemetry/instrumentation-xml-http-request': {
                propagateTraceHeaderCorsUrls: /.*/
              }
            }),
            // User events
            new UserInteractionInstrumentation({
              eventNames: ['click', 'input'] // instrument click and input events only
            }),
            // Custom Web Vitals instrumentation
            new WebVitalsInstrumentation({}, resource)
          ]
        });
```

```
// Initializer
public start() {
    if (!this.initialized) {
        // Enable automatic span generation for document load and user click interactions
        registerInstrumentations({
            instrumentations: [
                // Automatically tracks when the document loads
                new DocumentLoadInstrumentation({
                    ignoreNetworkEvents: false,
                    ignorePerformancePaintEvents: false
                }),
                getWebAutoInstrumentations({
                    '@opentelemetry/instrumentation-fetch': {
                        propagateTraceHeaderCorsUrls: /.*/,
                        clearTimingResources: true
                    },
                    '@opentelemetry/instrumentation-xml-http-request': {
                        propagateTraceHeaderCorsUrls: /.*/
                    }
                }),
                // User events
                new UserInteractionInstrumentation({
                    eventNames: ['click', 'input'] // instrument click and input events only
                }),
                // Custom Web Vitals instrumentation
                new WebVitalsInstrumentation({}, resource)
            ]
        });
```

otel-record-store / Applications / ... / records-ui-web / Transactions / HTTP GET

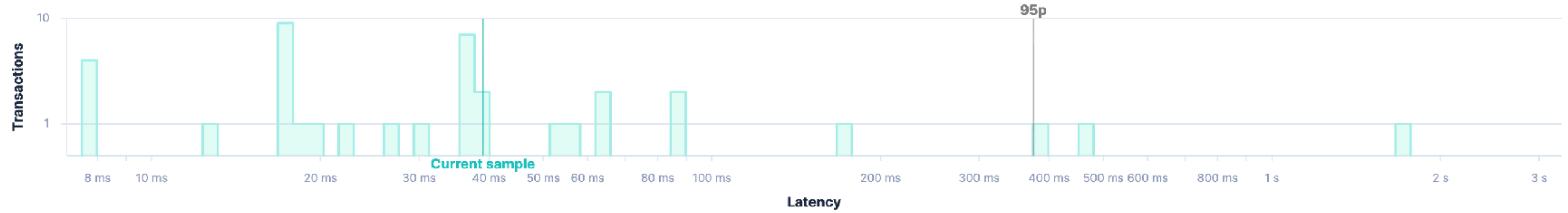
Give feedback | AI Assistant | CR

Give feedback | Alerts | Settings | Add data

Latency distribution

38 total transactions

Click and drag to select a range



All transactions

Trace sample

1 of 38

Investigate | View full trace

17 seconds ago | 39 ms (100% of trace)

Timeline | Metadata | Logs

Show critical path

Services records-ui-web record-store-server-java

0 ms | 5.0 ms | 10 ms | 15 ms | 20 ms | 25 ms | 30 ms | 35 ms | 39 ms

1 HTTP GET 39 ms

1 HTTP 2xx GET /records/featured 2.1 ms

GET featured 582 μs

```
    // Initializer
    public start() {
        if (!this.initialized) {
            // Enable automatic span generation for document load and user click interactions
            registerInstrumentations({
                instrumentations: [
                    // Automatically tracks when the document loads
                    new DocumentLoadInstrumentation({
                        ignoreNetworkEvents: false,
                        ignorePerformancePaintEvents: false
                    }),
                    getWebAutoInstrumentations({
                        '@opentelemetry/instrumentation-fetch': {
                            propagateTraceHeaderCorsUrls: /.*/,
                            clearTimingResources: true
                        },
                        '@opentelemetry/instrumentation-xml-http-request': {
                            propagateTraceHeaderCorsUrls: /.*/
                        }
                    }),
                    // User events
                    new UserInteractionInstrumentation({
                        eventNames: ['click', 'input'] // instrument click and input events only
                    }),
                    // Custom Web Vitals instrumentation
                    new WebVitalsInstrumentation({}, resource)
                ]
            });
```

Latency

All transactions

Trace sample 1 of 9

Investigate View full trace

a month ago | 230 ms (100% of trace)

Timeline Metadata Logs

Show critical path

Type records-ui-web http

	0 s	50 s	100 s	150 s	200 s	250 s	300 s	350 s	387 s
7	Success **documentLoad** 230 ms								
								resourceFetch 10 ms	
								resourceFetch 28 ms	
								resourceFetch 22 ms	
								resourceFetch 24 ms	
								resourceFetch 4.2 ms	
								resourceFetch 11 ms	
								resourceFetch 8.7 ms	


otel-record-store
Applications
...
records-ui-web
Transactions
documentLoad
Give feedback
AI Assistant
CR
All transactions
Trace sample
1 of 9
a month ago | 230 ms (100% of trace)
Timeline
Metadata
Logs
Show critical path
Type records-ui-web http
0 s
50 s
7
Success documentLoad 230 ms
Span details
View span in Discover
Name
resourceFetch
Dependency
localhost:4173
Service
records-ui-web
Transaction
documentLoad
a month ago | 11 ms (0.0% of transaction) | external http
Metadata
Filter...
How to add labels and other data
@timestamp
@timestamp 2025-07-25T12:09:58.941Z
agent
agent.name otlp
agent.version unknown
attributes
attributes.event.outcome success
attributes.event.success_count 1
attributes.http.response_content_length 13262
attributes.http.url http://localhost:4173/src/app.css?t=1753445398940
attributes.processor.event span
attributes.service.target.name localhost:4173
attributes.service.target.name.text localhost:4173
attributes.service.target.type http

```
    // Initializer
    public start() {
        if (!this.initialized) {
            // Enable automatic span generation for document load and user click interactions
            registerInstrumentations({
                instrumentations: [
                    // Automatically tracks when the document loads
                    new DocumentLoadInstrumentation({
                        ignoreNetworkEvents: false,
                        ignorePerformancePaintEvents: false
                    }),
                    getWebAutoInstrumentations({
                        '@opentelemetry/instrumentation-fetch': {
                            propagateTraceHeaderCorsUrls: /.*/,
                            clearTimingResources: true
                        },
                        '@opentelemetry/instrumentation-xml-http-request': {
                            propagateTraceHeaderCorsUrls: /.*/
                        }
                    }),
                    // User events
                    new UserInteractionInstrumentation({
                        eventNames: ['click', 'input'] // instrument click and input events only
                    }),
                    // Custom Web Vitals instrumentation
                    new WebVitalsInstrumentation({}, resource)
                ]
            });
```

D / otel-record-store / Applications / ... / records-ui-web / Transactions / click

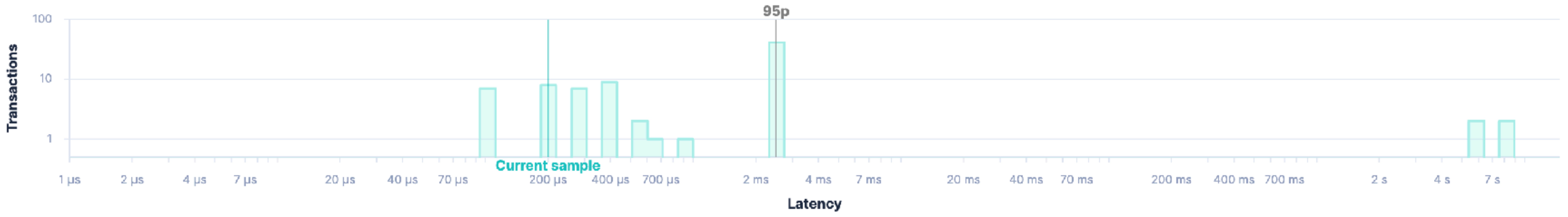
Give feedback | AI Assistant | CR

Give feedback | Alerts | Settings | Add data

Trace samples | Latency correlations | Failed transaction correlations

Latency distribution | 85 total transactions

Click and drag to select a range



All transactions

Trace sample 1 of 85

Investigate | View full trace

19 minutes ago | 200 µs (100% of trace)

Timeline | Metadata | Logs

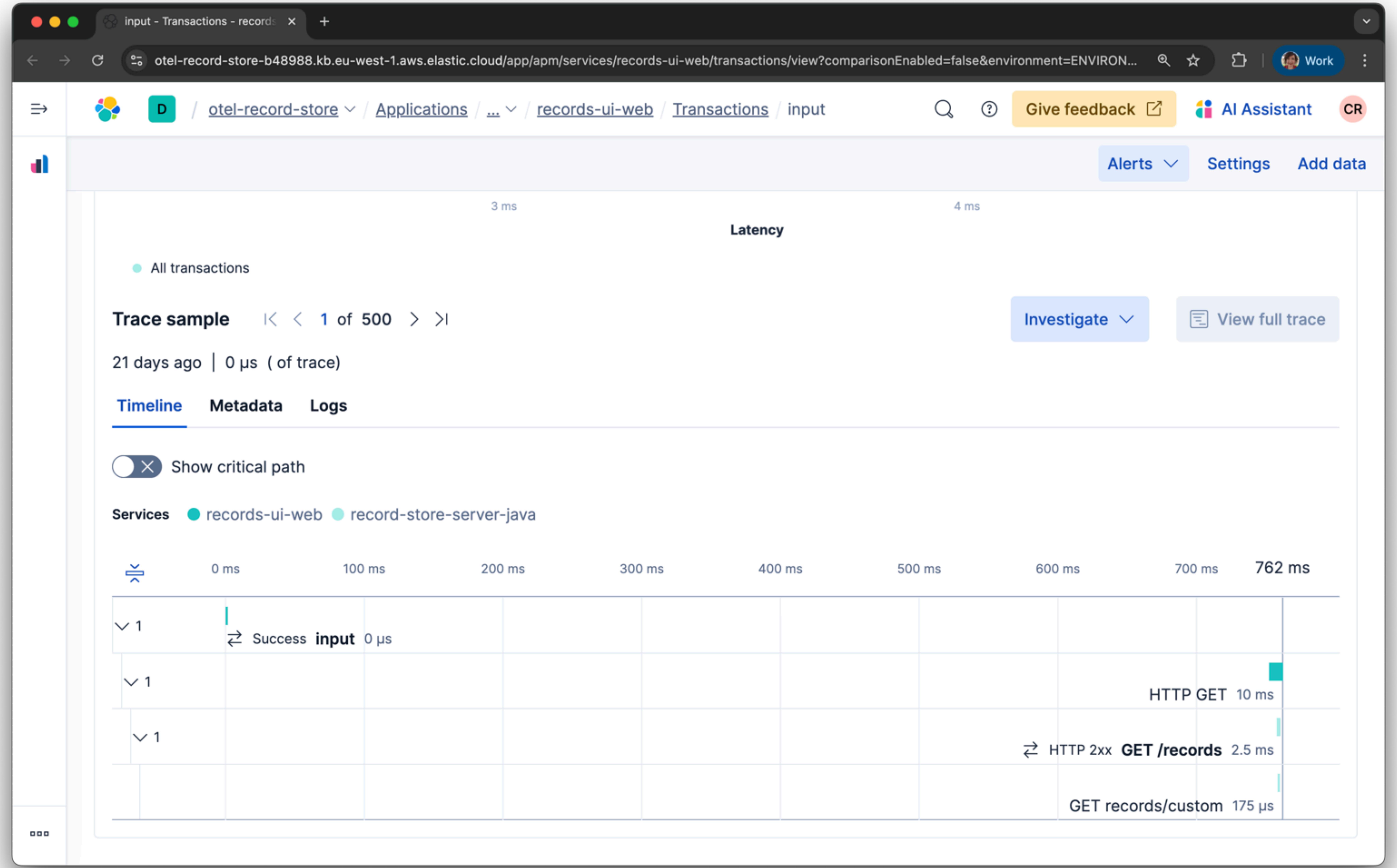
Show critical path

Type records-ui-web

0 µs	20 µs	40 µs	60 µs	80 µs	100 µs	120 µs	140 µs	160 µs	180 µs	200 µs

Success **click** 200 µs

D
otel-record-store
Applications
...
records-ui-web
Transactions
click
Give feedback
AI Assistant
CR
Give feedback
Alerts
Settings
Add data
Trace samples
Latency correlations
Failed transaction correlations
Latency distribution
85 total transactions
Click and drag to select a range
95p
100
10
1
Transactions
Current sample
1 μs
2 μs
4 μs
7 μs
20 μs
40 μs
70 μs
200 μs
400 μs
700 μs
2 ms
4 ms
7 ms
20 ms
40 ms
70 ms
200 ms
400 ms
700 ms
2 s
4 s
7 s
Latency
All transactions
Trace sample
1 of 85
Investigate
View full trace
20 minutes ago | 200 μs (100% of trace)
Timeline
Metadata
Logs
xpath
How to add labels and other data
attributes
attributes.target_xpath
//html/body/div/section/div/button
target_xpath
target_xpath
//html/body/div/section/div/button


input - Transactions - records
otel-record-store-b48988.kb.eu-west-1.aws.elastic.cloud/app/apm/services/records-ui-web/transactions/view?comparisonEnabled=false&environment=ENVIRON...
Work
D
otel-record-store
Applications
...
records-ui-web
Transactions
input
Give feedback
AI Assistant
CR
Alerts
Settings
Add data
3 ms
4 ms
Latency
All transactions
Trace sample
1 of 500
Investigate
View full trace
21 days ago | 0 μs (of trace)
Timeline
Metadata
Logs
Show critical path
Services
records-ui-web
record-store-server-java
0 ms
100 ms
200 ms
300 ms
400 ms
500 ms
600 ms
700 ms
762 ms
1
Success input 0 μs
1
HTTP GET 10 ms
1
HTTP 2xx GET /records 2.5 ms
GET records/custom 175 μs

JavaScript | OpenTelemetry

https://opentelemetry.io/docs/languages/js/

OpenTelemetry — Docs · Ecosystem · Status · Community · Blog · English · Search this site...

JavaScript

A language-specific implementation of OpenTelemetry in JavaScript (for Node.js & the browser).

This is the OpenTelemetry JavaScript documentation. OpenTelemetry is an observability framework – an API, SDK, and tools that are designed to aid in the generation and collection of application telemetry data such as metrics, logs, and traces. This documentation is designed to help you understand how to get started using OpenTelemetry JavaScript.

Status and Releases

The current status of the major functional components for OpenTelemetry JavaScript is as follows:

Traces	Metrics	Logs
Stable	Stable	Development

For releases, including the latest release, see Releases.

Warning

Client instrumentation for the browser is **experimental** and mostly **unspecified**. If you are interested in helping out, get in touch with the Client Instrumentation SIG.

View page source
Edit this page
Create documentation issue

Status and Releases
Version Support
Repositories
Help or Feedback

Source: Ruben. on Unsplash

Logs

Metrics

Traces

Source: Wilhelm Gunkel on Unsplash

DANKE!
THANK YOU!
MERCI!
GRAZIE!
GRACIAS!
DANK JE WEL!

SCAN ME